

Redundant Array Computation Elimination

ZIXUAN WANG, Institute of Computing Technology, Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, China

LIANG YUAN*, Institute of Computing Technology, Chinese Academy of Sciences, China

XIANMENG JIANG, Institute of Computing Technology, Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, China

KUN LI, Institute of Computing Technology, Chinese Academy of Sciences, China

JUNMIN XIAO, Institute of Computing Technology, Chinese Academy of Sciences, China

YUNQUAN ZHANG*, Institute of Computing Technology, Chinese Academy of Sciences, China

Redundancy elimination is a key optimization direction, and loop nests are the main optimization target in modern compilers. Previous work on redundancy elimination of array computations in loop nests either targets specific computation patterns or fails to recognize redundancies with complex structures. This paper proposes RACE (Redundant Array Computation Elimination), a hash-based technique that utilizes a novel two-level scheme to identify the data reuse between array references and the computation redundancies between expressions, enabling hierarchical redundancy detection beyond pattern-specific methods. It traverses the expression trees in loop nests to detect redundancies hierarchically in linear time and generates efficient code with optimized auxiliary arrays that store redundant computation results. Furthermore, RACE supports the expression reassociation with various aggressive strategies to improve the redundancy opportunities. Experimental results demonstrate the effectiveness of RACE.

Additional Key Words and Phrases: Redundancy elimination, array computation, loop nests

1 Introduction

Redundancy elimination is one of the key optimizations in modern compilers [2, 17]. Generally, redundancies fall into two types, unnecessary data movement and redundant computation. The first type includes peephole optimization, copy propagation, and dead-store elimination. The second type contains lazy code motion, dead code elimination, constant folding, common subexpression elimination, value numbering [10, 11, 18, 19, 25, 28]. Most techniques in both types target scalar variables and employ the data-flow analysis.

The loop nest transformation is another critical branch of optimizations. It reschedules the computation in the iteration space of the loop nest, such as loop fusion, loop skewing, loop unrolling, vectorizing, and loop tiling [1, 7, 31]. The primary purpose of these transformations is to improve the data locality and/or expose loop parallelism. In addition, there are several techniques to eliminate redundancies in loop nests. The loop-invariant code motion and induction variable elimination remove redundant computations [2, 26], but they still focus on scalar variables. The scalar replacement and array contraction target arrays, but they only reduce the array reference overhead.

*Corresponding author.

Authors' Contact Information: Zixuan Wang, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China and University of Chinese Academy of Sciences, Beijing, China, wangzixuan22@mails.ucas.ac.cn; Liang Yuan, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, yuanliang@ict.ac.cn; Xianmeng Jiang, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China and University of Chinese Academy of Sciences, Beijing, China, jiangxianmeng23z@ict.ac.cn; Kun Li, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, likungw@gmail.com; Junmin Xiao, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, xiaojunmin@ict.ac.cn; Yunquan Zhang, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, zyzq@ict.ac.cn.

Computation redundancies widely exist in loop nests of real-world applications. Furthermore, the redundancies are often related to arrays and across loop iterations, which is beyond the scope of the above-mentioned classic compiler optimization techniques. Several techniques are developed to exploit this kind of redundancy. Cooper et al. [12] propose the first redundancy elimination method for loops. Other methods [5, 22, 23] target specific computation patterns only. A detailed analysis is present in Section 3.

Techniques related to arrays usually rely on data dependence analysis [15, 21, 24]. The data dependence detection is generally undecidable since the addresses can be arbitrary functions. Although most array references are expressed by affine functions of loop index variables, solving the dependence precisely over the integer domain still constitutes an NP-hard problem, Diophantine equations [4].

This paper proposes RACE, a novel redundancy elimination method for array computations across loop iterations. Unlike previous work, which often utilizes data dependence analysis to explore the redundancy, RACE distinguishes between the principles of data dependence for loop transformation and data reuse for redundancy detection. Then it utilizes a novel two-level scheme to identify the array reference reuse and the expression computation redundancy. Redundancies can be easily discovered by grouping the expressions with the same identification value.

RACE traverses the expression trees in loop nests to detect redundancies hierarchically. Previous work usually requires pairwise comparisons between expressions, resulting in $O(n^2)$ time complexity for n expressions. However, RACE can complete the detection on the expression trees in linear time based on the two-level identification scheme. It generates efficient redundancy-eliminated codes with optimized auxiliary arrays that store redundant computing results. The code preserves the original order in the expression evaluation by only considering redundancies between binary operations, i.e. binary expression trees. Thus the result consistency is guaranteed.

Furthermore, RACE supports expression reassociation (rearranging operands to expose more opportunities for redundancy elimination) with various aggressive strategies to enhance redundancy detection. This is equivalent to detecting redundancies on n -ary expression trees, where the computation order can be reorganized. This paper makes the following contributions:

- A novel two-level scheme to identify the array references reuse and the expressions computation redundancy.
- A linear-time algorithm for detecting hierarchical redundancies in binary expression trees with guaranteed floating-point accuracy.
- An algorithm supporting expression reassociations for discovering more redundancies on n -ary trees.

The remainder of this paper is organized as follows. Section 2 presents an example to explain the basic idea of this work. Section 3 reviews related work. Section 4 gives an overview of RACE framework. Section 5 introduces the two-level identification scheme for finding array reference reuse and expression computation redundancy. Section 6 describes the linear redundancy detection algorithm on binary expression trees based on the two-level identification scheme and the optimization of the auxiliary array organization. Section 7 provides the enhanced algorithm focusing on n -ary expression trees and the simplification of the detection strategy. Section 8 discusses the implementation and limitations of RACE. Section 9 presents the evaluation and Section 10 concludes the paper.

2 Motivation

This section illustrates the fundamental idea of this work through an example. The left code in Figure 1 presents a loop nest (in Fortran language) from the Los Alamos National Lab's Parallel

<pre> DO j=2,ny DO i=2,nx zsw=cos(ulat(i-1,j-1)) xsw=cos(ulon(i-1,j-1))*zsw ysw=sin(ulon(i-1,j-1))*zsw zsw=sin(ulat(i-1,j-1)) zs=cos(ulat(i,j-1)) xs=cos(ulon(i,j-1))*zs ys=sin(ulon(i,j-1))*zs zs=sin(ulat(i,j-1)) zw=cos(ulat(i-1,j)) xw=cos(ulon(i-1,j))*zw yw=sin(ulon(i-1,j))*zw zw=sin(ulat(i-1,j)) zc=cos(ulat(i,j)) xc=cos(ulon(i,j))*zc yc=sin(ulon(i,j))*zc zc=sin(ulat(i,j)) tx(i,j)=p25*(xc+xsw+xw) ty(i,j)=p25*(yc+ys+yw+ysw) tz(i,j)=p25*(zc+zs+zw+zsw) END DO END DO </pre>	<pre> DO j=2,ny zs=cos(ulat(1,j-1)) zc=cos(ulat(1,j)) prevYsum=(zc*sin(ulon(1,j)) +(zs*sin(ulon(1,j-1))) prevZsum=sin(ulat(1,j)) +sin(ulat(1,j-1)) prevXsum=(zc*cos(ulon(1,j)) +(zs*cos(ulon(1,j-1))) DO i=2,nx zs=cos(ulat(i,j-1)) zc=cos(ulat(i,j)) xsum=(zc*cos(ulon(i,j)) +(zs*cos(ulon(i,j-1))) ysum=(zc*sin(ulon(i,j)) +(zs*sin(ulon(i,j-1))) zsum=sin(ulat(i,j)) +sin(ulat(i,j-1)) tx(i,j)=p25*(xsum+prevXsum) ty(i,j)=p25*(ysum+prevYsum) tz(i,j)=p25*(zsum+prevZsum) prevYsum=ysum prevZsum=zsum prevXsum=xsum END DO END DO </pre>	<pre> DO j=1,ny DO i=1,nx aa_0_0(i,j)=cos(ulon(i,j)) aa_0_1(i,j)=cos(ulat(i,j)) aa_0_2(i,j)=sin(ulon(i,j)) aa_0_3(i,j)=sin(ulat(i,j)) END DO END DO DO j=2,ny DO i=2,nx zsw=aa_0_1(i-1,j-1) xsw=aa_0_0(i-1,j-1)*zsw ysw=aa_0_2(i-1,j-1)*zsw zsw=aa_0_3(i-1,j-1) zs=aa_0_1(i,j-1) xs=aa_0_0(i,j-1)*zs ys=aa_0_2(i,j-1)*zs zs=aa_0_3(i,j-1) zw=aa_0_1(i-1,j) xw=aa_0_0(i-1,j)*zw yw=aa_0_2(i-1,j)*zw zw=aa_0_3(i-1,j) zc=aa_0_1(i,j) xc=aa_0_0(i,j)*zc yc=aa_0_2(i,j)*zc zc=aa_0_3(i,j) tx(i,j)=p25*(xc+xsw+xw) ty(i,j)=p25*(yc+ys+yw+ysw) tz(i,j)=p25*(zc+zs+zw+zsw) END DO END DO </pre>
--	---	--

Fig. 1. An example from the Parallel Ocean Program code. Left: the original code. Middle: optimized code by ESR. Right: code generated by RACE after one iteration.

<pre> DO j=1,ny DO i=1,nx aa_0_0(i,j)=cos(ulon(i,j)) aa_0_1(i,j)=cos(ulat(i,j)) aa_0_2(i,j)=sin(ulon(i,j)) aa_0_3(i,j)=sin(ulat(i,j)) aa_1_0(i,j)=aa_0_0(i,j)*aa_0_1(i,j) aa_1_1(i,j)=aa_0_2(i,j)*aa_0_1(i,j) END DO END DO DO j=2,ny DO i=1,nx aa_1_2(i,j)=aa_0_3(i,j-1)+aa_0_3(i,j) aa_2_0(i,j)=aa_1_0(i,j-1)+aa_1_0(i,j) aa_2_1(i,j)=aa_1_1(i,j-1)+aa_1_1(i,j) END DO END DO DO j=2,ny DO i=2,nx tx(i,j)=p25*(aa_2_0(i,j)+aa_2_0(i-1,j)) ty(i,j)=p25*(aa_2_1(i,j)+aa_2_1(i-1,j)) tz(i,j)=p25*(aa_1_2(i,j)+aa_1_2(i-1,j)) END DO END DO </pre>	<pre> j0=0 j1=1 DO i=1,nx aa_0_1_i_j=cos(ulat(i,1)) aa_0_3(i,j0)=sin(ulat(i,1)) aa_1_0(i,j0)=cos(ulon(i,1))*aa_0_1_i_j aa_1_1(i,j0)=sin(ulon(i,1))*aa_0_1_i_j END DO DO j=2,ny DO i=1,nx aa_0_1_i_j=cos(ulat(i,j)) aa_0_3(i,j1)=sin(ulat(i,j)) aa_1_0(i,j1)=cos(ulon(i,j))*aa_0_1_i_j aa_1_1(i,j1)=sin(ulon(i,j))*aa_0_1_i_j aa_1_2(i)=aa_0_3(i,j0)+aa_0_3(i,j1) aa_2_0(i)=aa_1_0(i,j0)+aa_1_0(i,j1) aa_2_1(i)=aa_1_1(i,j0)+aa_1_1(i,j1) END DO DO i=2,nx tx(i,j)=p25*(aa_2_0(i)+aa_2_0(i-1)) ty(i,j)=p25*(aa_2_1(i)+aa_2_1(i-1)) tz(i,j)=p25*(aa_1_2(i)+aa_1_2(i-1)) END DO j1=1-j1 j0=1-j0 END DO </pre>
--	--

Fig. 2. An example from the Parallel Ocean Program code. Left: code generated by RACE after three iterations. Right: final code by RACE.

Ocean Program (POP). In each iteration of the loop, it involves 16 sin/cos calls, 11 multiplications and 9 additions. This original code incorporates the computation redundancy elimination inside the innermost loop. It employs temporary variables *zsw*, *zs*, *zw* and *zc* to hold the corresponding results that are used twice in successive codes. Thus it explicitly saves 4 redundant sin/cos calls. These redundancies can be automatically eliminated by classic common subexpression elimination or value numbering in modern compilers.

In addition to the redundancy in the basic block of the loop body, there are obviously other redundancies. ESR [12] is able to find the redundancies across iterations of the innermost loop. The code optimized by ESR is shown in the middle code listing of Figure 1. ESR identifies that, for example, $xw + xsw$ is the same as the $xc + xs$ computed in the previous iteration of the innermost loop. Thus it employs extra temporary variables, such as *prevXsum*, to store these redundant computations. If the three multiplications with the loop-invariant variable *p25* are ignored, ESR removes half redundancies in each iteration compared with the original code: sin/cos calls are reduced from 16 to 8, multiplications from 8 to 4, and additions from 9 to 6.

However, computation redundancies also exist across iterations of the outer loop. For example, the *zsw* value in iteration (i, j) is the same as *zw* value in iteration $(i, j - 1)$. Thus it is reasonable to store these values in extra auxiliary arrays. RACE is able to find all the redundancies in loop nests. RACE follows a hierarchical scheme. It detects the redundancies between binary expressions iteratively, replaces them with auxiliary arrays, and generates the precompute loop before the original loop. The right list in Figure 1 shows the code generated after one iteration of RACE. It allocates four new auxiliary arrays *aa_0_**, where 0 means the first iteration of RACE, and replaces the computation redundancies with corresponding auxiliary array loads.

RACE repeats the redundancy detection and finds all the redundancies after three iterations, as shown in the left code of Figure 2. Note that the three auxiliary arrays in the second level *aa_1_** belong to two precompute loops since their ranges differ. RACE reduces the cost to 4 sin/cos calls, 5 multiplications and 6 additions per iteration. The auxiliary arrays exhibit a hierarchical structure which is used by RACE to optimize the final code, as listed in the right code of Figure 2. Auxiliary arrays *aa_0_0*, *aa_0_1* and *aa_0_2* are only used by other auxiliary arrays in the same precompute loop. Thus they are removed or replaced with a scalar variable for multiple uses. *aa_0_3*, *aa_1_0* and *aa_1_1* can be reduced to arrays whose sizes in *j* dimension is 2, and *aa_1_2*, *aa_2_0* and *aa_2_1* can be reduced to one-dimensional arrays.

3 Related Work

3.1 Data Dependence

The data dependence test [3, 17, 32] targets two array references and at least one of them is a write. On the contrary, redundant computation means two or more expressions performing the same operation on the same array elements (or the same values). This difference is not critical, since dependence techniques can also be applied to identify two array reads accessing the same memory address. However, the two analyses differ in more essential aspects.

Specifically, the essential differences between the data dependence test and the computation redundancy elimination are three-fold. First, the dependence test seeks to check whether there is at least one iteration that the two references access the same memory address. However, the redundancy elimination in loops only targets a large number of common computations. Second, the practical general dependence test technique is often conservative in the sense that it may report a false dependence but must report all real data dependence. However, as an optimization technique, the redundancy elimination should ensure a potential performance improvement. Finally, the data dependence test focuses on array references, while the computation redundancy targets

expressions. Therefore, it needs a new method to determine the redundancy between two data reuses and between two expressions.

3.2 Redundancy Elimination

Two classic redundancy elimination techniques for the scalar computation are the value numbering and common subexpression elimination (CSE) [10, 11, 27]. The key idea underlying both methods is to record the identification of the operands. The value numbering assigns the same number to the variables that share the same value, while CSE utilizes the available expression in the data-flow analysis to recognize the operand change. However, they only target scalar variables.

Cooper et al. [12] propose the enhanced scalar replacement (ESR), the seminal redundancy elimination method for loops. ESR employs two classic optimization techniques, value numbering and classic scalar replacement (CSR) [6]. ESR has the following disadvantages. First, ESR considers recomputation only across the innermost loop, thereby failing to discover all redundancies. Second, ESR incorporates scalar replacement, which relies on advanced data dependence analysis. However, data dependence is not suitable for redundancy detection, which requires a more precise definition and method. Third, ESR is a two-step method that separates the dependence-based array partition and expression-level index alignment. This separation leads to unnecessary comparisons between expressions that are not redundant. Fourth, the affinity graph in ESR fails to identify redundant computations among arrays within the same array partition. Finally, ESR seeks to find the maximal computation redundancy in a single execution of the algorithm. However, as the example in Section 2 demonstrates, the computation redundancy may exhibit a hierarchical property.

Deitz et al. [14] proposed array subexpression elimination (ASE) for sum-of-product array expressions. Like ESR, it also aims to eliminate the redundancies over the innermost loop dimension. ASE depends on a specific framework called neighborhood tablet to search the redundancy. Basu et al. [5] targeted stencil computations and designed a specific optimization scheme. It employs extra arrays to buffer partial sums within the leading plane of the whole iteration space. However, it only focuses on simple stencils and reduces redundant computations effectively for stencils with symmetric coefficients. These methods lack generality and fail to process complex subexpressions, e.g., $A(i) * C(j, k)$. Chi and Cong [8] developed heuristic search-based reuse (HSBR) for stencil computations. They did observe the conflict between expressions but did not formalize it as we do.

RCE (redundant computation elimination) [33] is able to explore redundancies with temporal locality for stencils but their search algorithm compares all possible expressions and incurs high overhead. This scheme is also utilized by the loop-carried CSE method proposed in [20]. On the contrary, our method assigns each expression a hash number and redundancies can be easily identified by extracting expressions with the same hash number. Wen et al. [30] implemented a profiler to pinpoint redundant computations by runtime checking. It finds redundancies in several stencil computations and eliminates them by hand-written optimizations. Ding et al. [15] proposed GLORE, which mainly focuses on loop-invariant reductions. For example, it is able to identify the redundant reduction $\sum_k B(j, k)$ in $\sum_{i,j,k} A(i, j)B(j, k)$. In contrast, RACE primarily targets the loop-carried redundant expressions, e.g. $\cos(ulat(i, j))$ in iteration (i, j) is identical to $\cos(ulat(i, j - 1))$ in iteration $(i, j + 1)$.

Redundancy elimination, as well as many other compiler optimizations, such as loop transformation, memory access optimization and vectorization, relies on the results of aliasing analysis [13, 16]. Alias analysis disambiguates memory references, primarily serving for the legality verification or candidate selection. For example, existing work like ESR [12] selects the redundancy candidates using the results of data dependence analysis which implicitly incorporates alias analysis. Thus, alias analysis is orthogonal to these optimizations to some extent. Furthermore, existing aliasing

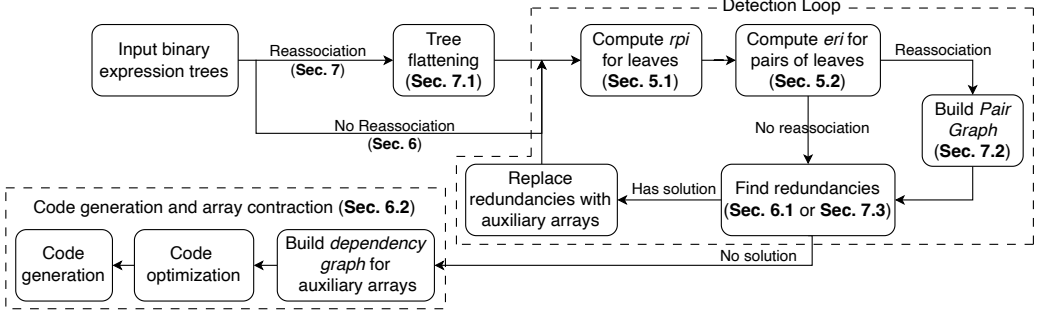


Fig. 3. Framework of the RACE method. "No reassociation" and "reassociation" correspond to the detection procedures on binary tree (Section 6) and n-ary tree (Section 7), respectively.

analysis techniques are powerful but not panaceas. Other mechanisms such as the restrict keyword, `-fno-alias` compiler option, and `#pragma ivdep` directive are provided to enable more aggressive optimizations. Therefore, in this paper, RACE focuses on the detection technique of redundancies, instead of directly incorporating alias analysis.

4 Overview

4.1 Scope and Assumptions

This paper focuses on perfectly nested loops without internal control flow. RACE eliminates redundant computations in unmodified arrays within the loop. Each candidate expression considered in the algorithm involves arithmetic operations on scalar or array variables. RACE targets array references of the form $A[a_1 i_{s_1} + b_1] \cdots [a_n i_{s_n} + b_n]$ (in the following text, array notations may appear in either C-style (e.g., $A[i][j]$) or Fortran-style (e.g., $A(i, j)$), depending on the context), where s_k is a loop level number ranking from the outermost loop to the innermost, $1 \leq s_k \leq m$, m is the nesting depth of the loop, i_{s_k} is a loop index, a_k and b_k are constants. For all the array references in the above POP example in Figure 1, we have $s_1 = 2$ and $s_2 = 1$, $i_{s_1} = i$ and $i_{s_2} = j$. This form covers most loop nests in real-world applications. For the applications used in the evaluation of this work, over 95% of array accesses in loops conform to this pattern. Scalars are treated as zero-dimensional array references. Function calls such as `sin` and `cos` can be interpreted as binary operators $\odot$, where the function name is treated as a scalar variable. For example, $\cos(\text{ulat}(1, j - 1))$ can be viewed as a binary expression $\cos \odot \text{ulat}(1, j - 1)$, where `cos` and $\text{ulat}(1, j - 1)$ are the left and right operands.

4.2 RACE Framework

Figure 3 illustrates the main components and overall workflow of RACE. The subsequent sections present the details of the components and algorithms involved. RACE takes binary expression trees inside a perfectly nested loop as input. The detection algorithm for binary tree (Section 6) follows the "no reassociation" path. It preserves the original evaluation order of expressions, thereby preserving floating-point results. The detection algorithm for n-ary tree (Section 7) follows the "reassociation" path, which restructures the input trees through tree flattening (Section 7.1) before the detection loop, exposing additional redundancies.

RACE detects redundancy iteratively. In each iteration of the detection loop, it identifies redundant binary subexpressions, replaces them with references to auxiliary arrays, and then continues on the transformed trees so that larger redundant expressions can be exposed in later iterations. This process is driven by a two-level hash-based identification scheme: RACE first computes

the reference pattern identifier rpi (Section 5.1) for array references and scalar leaves, and then computes the expression redundancy identifier eri (Section 5.2) for binary subexpressions. The binary-tree algorithm enables linear-time detection, whereas the n-ary-tree algorithm uses the Pair Graph (Section 7.2) to resolve conflicts among multiple candidate redundancies. After detection, RACE generates optimized code and then applies auxiliary-array optimization to reduce storage overhead and simplify the transformed program (Section 6.2).

5 Redundancy Identification

This section describes the two-level redundancy identification scheme. The first level defines a hash function for array references to identify data sharing. Based on the first level, the second level defines a hash function for binary expressions, and redundancies can be efficiently discovered by grouping the expressions with the same hash value.

5.1 Reference Pattern Identifier (rpi)

To recognize the redundancy among expressions, the data sharing among array references needs to be analyzed. Existing data dependence techniques test whether two array references may access at least one common array element. On the contrary, the redundancy elimination of array computations requires the reference of a sufficiently large set of common array elements. This paper introduces array *reference pattern* and only considers the redundancy among array references with the same reference pattern.

Intuitively, two array references share the same reference pattern only if they access the same infinite lattice in the iteration space, assuming that each loop index i_k ranges over $\mathbb{Z}$. For example, $A[i][j]$ and $A[i+1][j-1]$ both touch all the lattice points in $\mathbb{Z}^2$, while the reference lattices of $A[2i]$ and $A[2i+1]$ are disjoint, and $A[2i]$ and $A[3i]$ access partially overlapping lattice points.

Formally, an array reference of the form $A[a_1 i_{s_1} + b_1] \cdots [a_n i_{s_n} + b_n]$ accesses the infinite integer lattice $\mathcal{L}(\mathbf{B}, \mathbf{b}) = \{\mathbf{B}\mathbf{x} + \mathbf{b} \mid \mathbf{x} = (i_1, \dots, i_m) \in \mathbb{Z}^m\}$, where m is the nesting depth of the loop, n is the dimension of the array A , $\mathbf{B}$ is an $n \times m$ basis matrix with $B_{k,s_k} = a_k$ and all other entries equal to 0, and $\mathbf{b}$ is an n -dimensional offset vector whose k -th component is b_k . According to lattice theory, two array references access the same infinite lattice in $\mathbb{Z}^n$ if and only if there exists a unimodular matrix $\mathbf{U}$ such that $\mathbf{B}\mathbf{U} = \mathbf{B}'$ and $\mathbf{b} - \mathbf{b}' \in \mathcal{L}(\mathbf{B}, \mathbf{0})$, where $\mathbf{B}, \mathbf{b}$ and $\mathbf{B}', \mathbf{b}'$ denote their basis matrices and offset vectors.

A unimodular matrix can be represented as a product of elementary column operations, including: 1. column exchange ($A[i][j]$ to $A[j][i]$); 2. column multiplied by -1 ($A[i][j]$ to $A[-i][j]$); and 3. adding an integer multiple of one column to another ($A[i][j]$ to $A[i+2j][j]$). However, the reference patterns identified in this paper do not cover general unimodularly related lattices, for two practical reasons. First, the third operation does not appear since we have restricted the form of array subscripts previously, i.e. each row of $\mathbf{B}$ has only one nonzero entry. We also rarely detect redundancies that involve the first two operations in the evaluated applications. Second, although the unimodular condition is concise for infinite lattices, it is difficult in practice to decide whether two unimodularly related finite lattices overlap on a sufficiently large region [9], especially when loop bounds are unknown at compile time. Therefore, to simplify detection, we define the reference pattern as follows. Two array references have the same reference pattern if and only if their lattices $\mathcal{L}(\mathbf{B}, \mathbf{b})$ and $\mathcal{L}(\mathbf{B}', \mathbf{b}')$ satisfy: 1. $\mathbf{B} = \mathbf{B}'$, and 2. $\mathbf{b} - \mathbf{b}' \in \mathcal{L}(\mathbf{B}, \mathbf{0})$ (i.e., $\mathbf{B}\mathbf{x} = \mathbf{b} - \mathbf{b}'$ has an integral solution).

Checking these two conditions appears to require pairwise comparisons between array references. However, the information required by the two conditions can be encoded locally for each individual array reference. We introduce a hash function, called *reference pattern identifier* (rpi), to perform this encoding. If two array references have the same rpi value, they satisfy the above two conditions

and thus have the same reference pattern. The first condition ($\mathbf{B} = \mathbf{B}'$) is straightforward. It follows that if two references share the same reference pattern, their index list (dimension axis) $\langle i_{s_1}, \dots, i_{s_n} \rangle$ and the coefficient list $\langle a_1, \dots, a_n \rangle$ must be the same, respectively. Thus, rpi must incorporate these two lists.

The second condition ($\mathbf{b} - \mathbf{b}' \in \mathcal{L}(\mathbf{B}, \mathbf{0})$) needs more elaboration. Consider two potential references $A[a_1 i_{s_1} + b_1] \cdots [a_n i_{s_n} + b_n]$ and $A[a_1 i_{s_1} + b'_1] \cdots [a_n i_{s_n} + b'_n]$. If a loop index $i_{s_k} = i$ appears only once in the subscripts, $a_k i = b_k - b'_k$ has an integral solution if and only if $b_k \bmod a_k = b'_k \bmod a_k$. Thus, rpi incorporates $b_k \bmod a_k$. For example, $A[2i]$ and $A[2i + 2]$ share the same rpi. If i appears in multiple subscripts, the successive deltas (differences) must also match. Assuming $i_{s_k} = i_{s_j} = i$, the equations $a_k i = b_k - b'_k$ and $a_j i = b_j - b'_j$ must share the same integer solution. Besides the condition $b_k \bmod a_k = b'_k \bmod a_k$, it is also required that $b_k/a_k - b_j/a_j = b'_k/a_k - b'_j/a_j$. Therefore rpi should also incorporate $b_k/a_k - b_j/a_j$. For example, $A[2i + 1][3i + 2]$ and $A[2i + 3][3i + 5]$ have the same delta value $2/3 - 1/2 = 5/3 - 3/2 = 1/6$ for i . Thus they access the same infinite lattice.

Algorithm 1: Calculating information for rpi

```

input : An array reference  $x = A[a_1 i_{s_1} + b_1] \cdots [a_n i_{s_n} + b_n]$ 
output: indexList, indexCoef, indexDelta, firstIndexOffset of  $x$ 
// x.indexList and x.indexCoef are lists
// x.firstIndexOffset is an array of rationals
// x.indexDelta is an array of lists of rationals
x.firstIndexOffset[1...m] =  $\infty$ ; // m is the nesting depth of the loop
for  $k \leftarrow 1$  to  $n$  do
  if  $a_k \neq 0$  then
    x.indexList.append( $s_k$ );
    x.indexCoef.append( $a_k$ );
    if x.firstIndexOffset[ $s_k$ ] =  $\infty$  then
      x.firstIndexOffset[ $s_k$ ] =  $b_k/a_k$ ;
      x.indexDelta[ $s_k$ ].append( $b_k \bmod a_k$ );
    else
      x.indexDelta[ $s_k$ ].append( $b_k/a_k - x.firstIndexOffset[s_k]$ );
    end
  else
    x.indexList.append(0);
    x.indexCoef.append( $b_k$ );
  end
end
end

```

Algorithm 1 shows the pseudo-code that extracts the information for rpi. The descriptions of the variables used are given below: *indexList* and *indexCoef* store the loop index variables and their corresponding coefficients in the order they appear. *firstIndexOffset* stores the normalized offset of each loop index variable when it first appears. It is used to compute *indexDelta*, which contains the successive deltas of the same loop index variable. In calculating *indexDelta*, it keeps the rational results in irreducible fraction form. If a loop index is missing, i.e. $a_k = 0$, it appends a virtual loop level 0 and the corresponding constant b_k must be added to the coefficient list. For example, $A[i][1]$ and $A[i][2]$ do not share common elements, and the coefficients 1 and 2 are the only identifiers in

this dimension. Let $x.name$ denote the name of the array reference x . Putting it all together, rpi is formulated as follows:

$$rpi(x) = \text{hash}(x.name, x.indexList, x.indexCoef, x.indexDelta).$$

5.2 Expression Redundancy Identifier (eri)

With rpi of each operand, we define the expression redundancy identifier (eri) to recognize expressions with redundant computations. eri also uses the local-encoding idea to avoid pairwise comparisons. Expressions sharing the same eri value are considered redundant. Intuitively, rpi identifies potential reuse at the array-reference level, while eri further distinguishes whether two expressions align in their operand index shifts. eri cannot be as simple as the classical value number definition like $vn(e) = \text{hash}(vn(x), \oplus, vn(y))$. The expressions may still not represent redundant computations, even if the corresponding operands in two expressions share the same rpi values. For example, the two expressions $e = A[i] + B[i]$ and $e' = A[i+1] + B[i+2]$ are not redundant, although $rpi(A[i]) = rpi(A[i+1])$ and $rpi(B[i]) = rpi(B[i+2])$. Therefore, we introduce the expression-level delta value $exprDelta$. Algorithm 2 presents the pseudo-code for calculating this information for eri . $exprDelta$ stores the delta between the two operands of the expression for each loop index variable. For example, supposing the loop nest order is $\langle i, j, k \rangle$, for $e = A[i][2j+1] + B[2i+3][k]$, we have $A[i][2j+1].firstIndexOffset = \langle 0, 1/2, \infty \rangle$, $B[2i+3][k].firstIndexOffset = \langle 3/2, \infty, 0 \rangle$, thus $e.exprDelta = \langle -3/2, \infty, \infty \rangle$.

Algorithm 2: Calculating information for eri

input : An expression $e = x \oplus y$
output: $exprDelta$ of e
 // $e.exprDelta$ is an array of rationals
 $e.exprDelta[1 \cdots m] = \infty$;
for $s_k \in \{x.indexList\} \cap \{y.indexList\} - \{0\}$ **do**
 $e.exprDelta[s_k] = x.firstIndexOffset[s_k] - y.firstIndexOffset[s_k]$;
end

By incorporating $exprDelta$ into the hash, eri captures the expression-level alignment information and yields identical values only when both the operand reference patterns and their relative index offsets are the same, which ensures correctness for expression-level redundancy detection. The formula of eri for $e = x \oplus y$ is as follows:

$$eri(e) = \text{hash}(rpi(x), \oplus, rpi(y), e.exprDelta).$$

For commutative operators such as addition and multiplication, the two operands need to be sorted to recognize the redundancy. For example, $A[i] + B[i]$ and $B[i+1] + A[i+1]$ are considered as redundant computations. The two operands can be sorted by their *name* A and B lexicographically. If the two operands access the same array, they are sorted according to their other rpi information: *indexList*, *indexCoef* and *indexDelta*. Thus, the redundancies between $A[i] + A[2i]$ and $A[2i+2] + A[i+1]$, $A[i] + A[i+1]$ and $A[i+2] + A[i+1]$ can be found. Without loss of generality, we assume that the operands in $e = x \oplus y$ are already in order.

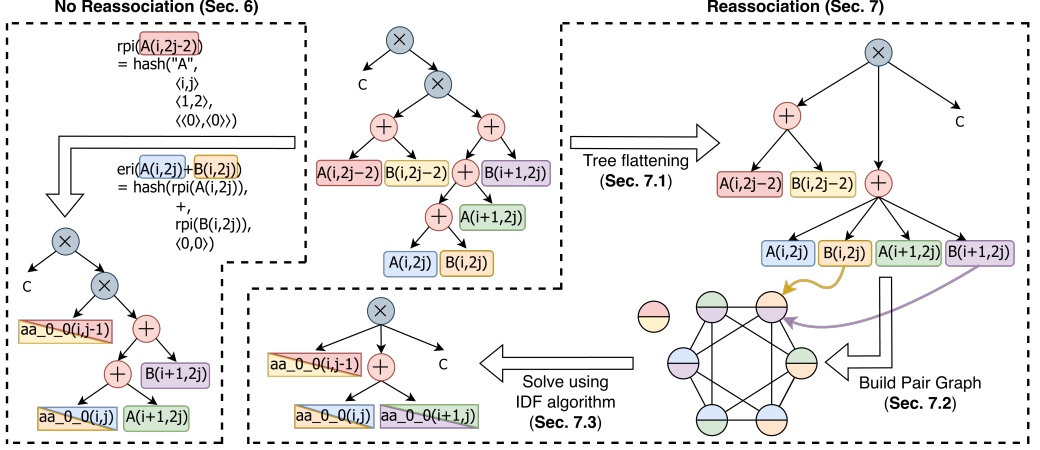


Fig. 4. Example of redundancy detection in RACE. The middle shows the input expression tree. The left illustrates the binary-tree detection algorithm (Section 6), which detects redundancies while preserving the original evaluation order. The right illustrates the n-ary-tree detection algorithm (Section 7), where tree flattening, Pair Graph construction, and the IDF algorithm enable the detection of additional redundancies. The detected redundancies are replaced with auxiliary array references, yielding different transformed expression trees.

6 Redundancy Elimination on Binary Trees

6.1 Detection Algorithm

This section presents the redundancy detection algorithm for binary expression trees. Two reasons motivate this choice of intermediate representation (IR). The first is to preserve result consistency, since binary trees fix the computation order and thus maintain identical numerical results under floating-point arithmetic. The second reason is that array redundancies naturally form a hierarchical dependence structure, which cannot be captured by a linear IR (e.g., three-address code). For example, in $a = B[\dots] \oplus C[\dots]$, the rpi values of the operands and the eri of a can be computed. When a subsequent statement $e = a \oplus D[\dots]$ appears, $rpi(a)$ is not informative for array-aware redundancy detection, because scalar a no longer carries the reference information of $B[\dots]$ and $C[\dots]$ until it is replaced by an auxiliary array $F[\dots]$. This contrasts with the classical value numbering method, which works on scalar expressions in three-address code IR. It can detect all redundancies in a single sequential traversal of the IR, since for a scalar assignment $a = b \oplus c$, the value number of a can be directly computed from those of b and c , and then used in subsequent redundancy checks.

The detection algorithm follows the "no reassociation" path in Figure 3, with an example shown in the left panel of Figure 4. It enters the detection loop without modifying the structure of the input tree. First, it computes the rpi values for all leaf nodes. Second, it assigns eri values to operator nodes whose two children are leaves. Expressions with the same eri value are then treated as redundant and replaced by new auxiliary array references. These new references must maintain consistent subscripts. To achieve this, one representative expression is selected for each auxiliary array. For example, $A(i,2j) + B(i,2j)$ and $A(i,2j-2) + B(i,2j-2)$ share the same eri , and an auxiliary array aa_0_0 is allocated for them. A representative expression is assigned to aa_0_0 , e.g., $aa_0_0(i,j) = A(i,2j) + B(i,2j)$. The other redundant expressions are replaced by references to this array with corresponding index offsets, e.g., $A(i,2j-2) + B(i,2j-2)$ will be replaced by

$aa_0_0(i, j - 1)$. Finally, the newly introduced auxiliary array references are treated as new leaves. The detection loop continues until no new redundancies are detected.

The algorithm detects and organizes redundancies hierarchically, as illustrated by the POP example in Section 2. In each iteration of the detection loop (except the first), new redundant expressions are discovered, whose operands include auxiliary arrays generated in the previous iteration. This iterative process is valid because redundancies are composable. For instance, if $(x \oplus y) \oplus z$ is redundant with $(x' \oplus y') \oplus z'$, it can be identified in the next iteration after $x \oplus y$ is first recognized as redundant and replaced with an auxiliary array.

The algorithm runs in linear time complexity: the redundancies can be identified in a single bottom-up traversal of the binary expression tree. The rpi values of the leaves and the eri values of the expressions are stored directly within the tree nodes. The proposed two-level hash scheme in Section 5 simplifies the redundancy detection procedure, so that the expressions sharing the same eri can be identified as redundant and replaced by auxiliary arrays references as new tree leaves. In the next iteration, only the newly introduced leaves and expressions need to be processed.

6.2 Code Generation and Array Contraction

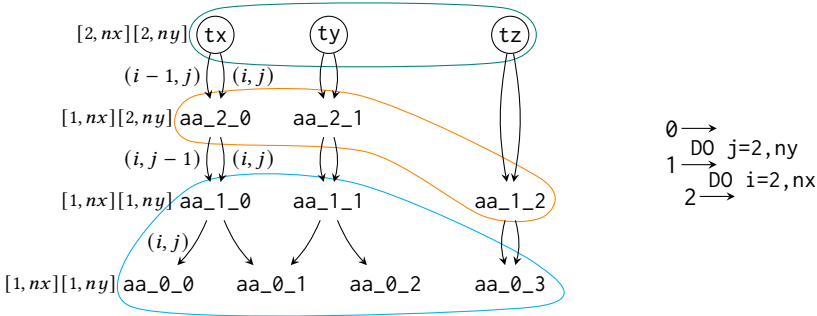


Fig. 5. Dependency graph of the auxiliary arrays in POP benchmark.

Code Generation. The expression trees are optimized with auxiliary arrays after the execution of RACE. Since the previous auxiliary arrays may be used by the newer ones during the iterative process, the dependencies between auxiliary arrays are represented by a directed acyclic graph (the *dependency graph*) with a natural topological order. Figure 5 illustrates the dependency graph of auxiliary arrays in POP benchmark.

To generate the straightforward code (left of Figure 2), we need (i) a topological order and (ii) loop ranges. The top nodes tx , ty , tz inherit the original loop range $[2, nx][2, ny]$. Using the representative expressions, each parent propagates its reference offsets to determine the child's range. Ranges are computed in topological order. For example, aa_2_0 is referenced at (i, j) and $(i - 1, j)$ by $tx(i, j)$, so its range becomes $[1, nx][2, ny]$. Figure 5 annotates nodes with their ranges and reference offsets.

Nodes with identical ranges are grouped into a *range circle* (colored circle in Figure 5); arrays in the same circle are initialized together by one loop. We then build a coarse graph over range circles, where each range circle corresponds to a node in the coarse graph, and generate loops in its topological order; inside a circle we still respect the fine-grained dependency order. This yields the code on the left of Figure 2.

Array Contraction. This simple code allocates a large space for the auxiliary array. However, it is possible to decrease the auxiliary dimensionality and reuse a small array space. Several contraction

rules are given as follows: First, if an auxiliary array has a single parent, inline its representative expression instead of storing it. In Figure 2, aa_0_0 and aa_0_2 are replaced by $\cos(ulon(i, j))$ and $\sin(ulon(i, j))$. Second, if an auxiliary array and all its parents are in the same range circle, all reference offsets must be zero, so it can be a scalar. Thus aa_0_1 becomes $aa_0_1_i_j$ (Figure 2, right). Third, the dimension of auxiliary arrays can be eliminated or compressed by inserting precompute loops at inner insertion positions (numbered in Figure 5, right). Treat each circle as a unit and, following the topological order, check positions from 0 inward. If a circle's range along dimension j matches that of the original loop, move it to position 1 and eliminate dimension j from all arrays inside it. The check can repeat inward until it fails. The orange circle satisfies this, so aa_2_0 , aa_2_1 , aa_1_2 lose dimension j (Figure 2, right). Note that the innermost dimension i is retained to facilitate vectorization. Even if the above dimension range check fails, the dimension can still be compressed. For example, the blue circle's j range $[1, ny]$ differs from the original $[2, ny]$ due to $j - 1$ references offset. Rather than placing the loop at position 0, we split it into a boundary prefetch $[1, 1]$ at position 0 and a main part $[2, ny]$ at position 1, allocate only two j -slices (j_0, j_1) and swap them each iteration, which is a double buffer of size $nx \times 2$. aa_0_3 , aa_1_0 , and aa_1_1 illustrate this situation (Figure 2, right).

6.3 Redundancy Analysis

After identifying the expressions with the same *eri* value, these redundant expressions can be replaced with precomputed auxiliary arrays. The range information for each auxiliary array is computed in the dependency graph. The evaluation of the profit gained from the replacement, i.e., the number of arithmetic operations eliminated, is discussed in this section.

Suppose that the auxiliary arrays $aa_1, \dots, aa_m$ appear in the transformed expression trees while the remaining positions are the original tree nodes. Let $cnt(aa_k)$ denote the number of occurrences of aa_k in the expression trees. For loop ranges, only the difference between the lower and upper bound is considered, which is denoted by $r(i_t)$ for the original loop range at index i_t . Let $r(i_t, aa_k)$ denote the precompute loop range of auxiliary array aa_k at index i_t .

The reduction in arithmetic operations is only related to the modified parts of the expression trees. The first step computes the original cost of the replaced parts. This is done by recursively expanding each auxiliary array's representative expression: if aa_k contains another auxiliary array aa_p , it is replaced by aa_p 's representative expression until no auxiliary arrays remain. Let the number of arithmetic operations in the expanded representative expression of aa_k be denoted by $ops(aa_k)$. The original computational cost of the positions replaced by the auxiliary arrays in the expression tree can be computed by:

$$ori = \prod_{t=1}^n r(i_t) \sum_{k=1}^m ops(aa_k) \cdot cnt(aa_k).$$

The generated code replaces the *ori* operations with auxiliary array reads. However, generating the auxiliary arrays requires precomputation loops. Therefore, the computational cost of this precomputation must also be considered to assess the benefit. Since our *eri* method targets binary expressions, each auxiliary array's precompute expression is a binary expression. Thus, the precomputation cost of the auxiliary arrays is:

$$aft = \sum_{k=1}^m \prod_{t=1}^n r(i_t, aa_k).$$

Hence, the profit gained from the detection algorithm on binary tree is $ori - aft$.

7 Redundancy Elimination on n-ary Trees

This section enhances redundancy detection for processing n-ary expression trees. The "reassociation" path in Figure 3 illustrates the redundancy detection on n-ary trees. The input binary expression trees are first transformed into n-ary trees before entering the detection loop. The motivation and strategies for this transformation are discussed in Section 7.1. In each iteration of the detection algorithm, the following steps are performed: compute rpi in the same way as before; for every operator node in the n-ary expression tree, compute eri for every pair of its leaf children (operands); build the Pair Graph (defined in Section 7.2) with eri as the color of each node; identify redundancies on the graph, with the objective function formulated in Section 7.2, and a heuristic method proposed in Section 7.3; finally, replace multiple redundant binary subexpressions with auxiliary arrays. The iteration continues until no further redundancy is found on the graph.

7.1 Tree Flattening

<pre> DO j=2,n-1 DO k=2,n-1 DO i=2,n-1 U(i,k,j)=U(i,k,j) +w0*R(i,k,j) +w1*(R(i-1,k,j)+R(i+1,k,j) +R(i,k-1,j)+R(i,k+1,j) +R(i,k,j-1)+R(i,k,j+1)) +w2*(R(i-1,k-1,j)+R(i+1,k-1,j) +R(i-1,k+1,j)+R(i+1,k+1,j) +R(i,k-1,j-1)+R(i,k+1,j-1) +R(i,k-1,j+1)+R(i,k+1,j+1) +R(i-1,k,j-1)+R(i-1,k,j+1) +R(i+1,k,j-1)+R(i+1,k,j+1)) +w3*(R(i-1,k-1,j-1)+R(i+1,k-1,j-1) +R(i-1,k+1,j-1)+R(i+1,k+1,j-1) +R(i-1,k-1,j+1)+R(i+1,k-1,j+1) +R(i-1,k+1,j+1)+R(i+1,k+1,j+1)) ENDDO ENDDO ENDDO </pre>	<pre> DO j=2,n-1 DO k=2,n-1 DO i=1,n aa_1_0(i)=R(i,k-1,j) +R(i,k+1,j) +R(i,k,j-1) +R(i,k,j+1) aa_1_1(i)=R(i,k-1,j-1) +R(i,k+1,j-1) +R(i,k-1,j+1) +R(i,k+1,j+1) ENDDO DO i=2,n-1 U(i,k,j)=U(i,k,j) +w0*R(i,k,j) +w1*(R(i-1,k,j) +R(i+1,k,j) +aa_1_0(i)) +w2*(aa_1_0(i-1) +aa_1_0(i+1) +aa_1_1(i)) +w3*(aa_1_1(i-1) +aa_1_1(i+1)) ENDDO ENDDO ENDDO </pre>	<pre> DO j=2,n-1 DO k=2,n-1 DO i=1,n aa_1_0(i)=R(i,k-1,j) +R(i,k+1,j) +R(i,k,j-1) +R(i,k,j+1) aa_1_1(i)=R(i,k-1,j-1) +R(i,k+1,j-1) +R(i,k-1,j+1) +R(i,k+1,j+1) aa_3_0(i)=w1*R(i,k,j) +w2*aa_1_0(i) +w3*aa_1_1(i) ENDDO DO i=2,n-1 U(i,k,j)=U(i,k,j) +w0*R(i,k,j) +w1*aa_1_0(i) +w2*aa_1_1(i) +aa_3_0(i-1) +aa_3_0(i+1) ENDDO ENDDO ENDDO </pre>
--	--	---

Fig. 6. An example from the mgrid benchmark.

Detecting redundancies in array or scalar computations may require *reassociation*, i.e., reorganizing arithmetic calculations using commutative, associative, or distributive laws. Consider $x + y + z$ and $x + z$. Compilers may evaluate the former as $((x + y) + z)$ (left-associative), but the redundancy with $x + z$ cannot be exposed unless reassociation is permitted. Note that the commutative law here differs from that used in eri calculation, where expressions have only two operands.

However, reassociation is generally illegal when result consistency must be preserved, even for integers. For instance, with large positive x, z but small negative y , both $((x + y) + z)$ and $(x + (y + z))$ may yield the correct value, whereas $((x + z) + y)$ may overflow. Nevertheless, compilers usually provide options enabling such transformations for aggressive optimizations.

Therefore, the implementation of RACE provides several options to control the reassociation scheme. We describe them using four aggressive levels. First, when consistency is required, reassociation is disabled and the linear redundancy detection algorithm is used. Second, parentheses in

source code are respected, and reorganization can only be applied inside each pair of parentheses. Third, parentheses are removed when the inside operators are consistent with the one outside the parentheses. Finally, the distributive law is applied to remove more parentheses.

The first three levels are relatively straightforward. Figure 4 illustrates an example of flattening an expression tree using the third level, where the nodes $A(i, 2j)$, $B(i, 2j)$, $A(i + 1, 2j)$, and $B(i + 1, 2j)$ share the same parent node. The redundant expression $A(i + 1, 2j) + B(i + 1, 2j)$ is exposed, which can not be identified in the original input tree. However, the distributive law should be applied cautiously as it may destroy existing redundancies. For example, in $e = (A(i) + B(i)) * (C(i) + D(i)) + (A(i + 1) + B(i + 1)) * (C(i + 2) + D(i + 2))$, two redundancies exist: $\text{eri}(A(i) + B(i)) = \text{eri}(A(i + 1) + B(i + 1))$ and $\text{eri}(C(i) + D(i)) = \text{eri}(C(i + 2) + D(i + 2))$. However, if the distributive law is applied, both of the redundancies disappear since none of the binary multiplication expressions has the same eri. Moreover, the distributive law may incur more computations. For example, there are 2 multiplications and 5 additions in e , but 8 multiplications and 7 additions after removing the parentheses. The detailed discussion is beyond the scope of this paper.

In practice, the distributive law is applied only when multiplying by a constant or loop-invariant scalar, which may yield additional redundancies. Figure 6 shows a 3D stencil loop in the SPEC2000 mgrid benchmark. The middle part shows the result at the second aggressive level, where auxiliary arrays aa_1_0 and aa_1_1 are found in the second iteration based on aa_0_* . Using the fourth strategy reveals new redundancies, producing a new auxiliary array aa_3_0 in the fourth iteration.

Another set of options can specify the reassociation for subtraction and division operations. For example, $x - y - z$ yields binary subexpressions including $x - y$, $x - z$, and $-y - z$. To identify redundancy between $y + z$ and $-y - z$, subtractions are rewritten as addition with negated operands (e.g., $-y - z \rightarrow (-y) + (-z)$), and the first operand in eri computation is standardized to "+". The division operation can be handled similarly.

7.2 Problem Formulation and Transformation

The algorithm on n-ary expression trees follows the iterative scheme as on binary trees. However, redundancy identification on n-ary trees is more complex than on binary trees, where eri values are computed directly for each binary expression and redundancies are detected when the eri values are identical. In an n-ary tree, more than two operands may share the same parent operator node, and their evaluation order can be rearranged to eliminate more redundancies. Therefore, multiple binary subexpressions may be identified as redundant within an n-ary expression, and some of them can conflict with each other. Specifically, the expression $e = x \oplus y \oplus \dots$ in an n-ary tree is denoted by $\oplus(x, y, \dots)$. If both $x \oplus y$ and $y \oplus z$ in the expression $\oplus(x, y, z)$ are redundant, only one of them can belong to the solution.

To model the conflict relationship between binary subexpression candidates. We define the *Pair Graph*, an undirected conflict graph $G = (V, E)$ to formalize the constraint. V contains all possible binary subexpressions. In particular, it generates $\binom{n}{2}$ binary subexpressions for each operator node with n leaf children. A node and an expression are equivalent in G and used interchangeably. Each edge indicates the conflict relation between two nodes, which means that the two expressions (nodes) share the same array reference and cannot be extracted simultaneously. Thus, any legal solution is an independent set of G . Figure 4 shows the corresponding Pair Graph to the flattened tree. The formal definition of G is as follows:

$$V = \bigcup_{e=\oplus(\dots)} \bigcup_{x, y \in e} \{x \oplus y \mid x, y \text{ are leaves}, x \neq y\},$$

$$E = \{(v, v') \mid v, v' \in V, v = x \oplus y \neq v' = x' \oplus y', \{x, y\} \cap \{x', y'\} \neq \emptyset\}.$$

In each iteration of the detection algorithm, finding redundancies corresponds to selecting a set of non-conflicting binary subexpression candidates, which forms an independent set of the Pair Graph. To determine the optimal solution, an objective function on the graph needs to be defined. The loop ranges are usually not known at the compiling time. So it cannot precisely evaluate the redundant computations between conflict redundancies. For example, given three expressions $\oplus(x, y, z)$, $\oplus(x', y')$ and $\oplus(y'', z'')$, if $\text{eri}(x \oplus y) = \text{eri}(x' \oplus y')$ and $\text{eri}(y \oplus z) = \text{eri}(y'' \oplus z'')$, it is hard to determine which one incurs more redundant computations before the execution.

Therefore, we use an approximate metric. If n expressions with the same eri are extracted, the new expression trees have n fewer binary expressions that are replaced with auxiliary array loads. The reduced number of redundant computations on the expression trees can be approximated to $n - 1$, where the minus one denotes the precomputation of the auxiliary array. Let $S \subseteq V$ be a solution and $\text{eri}(S)$ be the set of all eri values of expressions in S , i.e.,

$$\text{eri}(S) = \{\text{eri}(v) \mid v \in S\}.$$

The optimal solution in a single detection iteration is tied to the objective function:

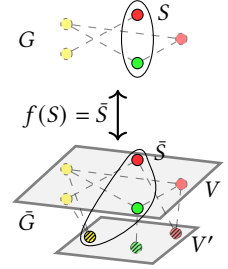
$$\underset{S \in I_G}{\text{argmax}} |S| - |\text{eri}(S)|, \quad (1)$$

where I_G denotes the set of all independent sets of graph G .

This problem can be reduced to the Maximum Independent Set (MIS) problem by augmenting the Pair Graph G to $\bar{G} = (\bar{V}, \bar{E})$. For each eri value k , $\bar{G}$ adds an auxiliary node $\bar{v}_k$ connected to all nodes with eri value k in G , i.e., $\bar{V} = V \cup V'$ where $V' = \{\bar{v}_k \mid k \in \text{eri}(V)\}$, $\text{eri}(\bar{v}_k) = k$, and $\bar{E} = E \cup \{(\bar{v}_k, v) \mid \bar{v}_k \in V', v \in V, \text{eri}(v) = k\}$. The formal description is in Theorem 7.1.

THEOREM 7.1. *The optimization problem on graph G defined in Equation 1 can be reduced to the MIS problem on $\bar{G}$.*

Proof sketch. We first define a bijective function f that maps each independent set S of G to an independent set $\bar{S}$ of $\bar{G}$ by adding auxiliary nodes. An illustration is shown on the right. Then we show that every MIS of $\bar{G}$ lies in the range of f , which establishes a correspondence between the solution space of the original problem on G and that of the MIS problem on $\bar{G}$. Finally, two objective functions are introduced for the optimization problems on G and $\bar{G}$, respectively, and we prove their equivalence under f . The full proof is provided in Appendix A.



7.3 Inner-Dimension-First Heuristic

It is well-known that the MIS problem is NP-hard. For a graph with n nodes, the worst-case time complexity is $O(2^n)$. For expression trees with numerous operands, exhaustive search quickly becomes computationally infeasible. In addition, Equation 1 on the Pair Graph may admit multiple optimal solutions. Different solutions correspond to different sets of auxiliary arrays, which in turn can lead to different performance of the generated code. Auxiliary arrays whose reuses occur along the innermost loop usually exhibit better locality and facilitate vectorization. We therefore introduce a heuristic strategy that reduces the problem size and improves the quality of the chosen auxiliary arrays by prioritizing expressions that satisfy specific conditions.

The array's memory access patterns in real applications are typically regular and symmetric. As described in Section 6.2, the auxiliary arrays can be compressed to lower dimensions to save memory and improve the cache performance. Based on the observation, we propose a heuristic algorithm called inner-dimension-first (IDF) extraction, which aims to find solutions with a higher

potential for compression while reducing the problem size. This approach increases the opportunity to leverage cache locality and vectorization. The intuition behind this approach can be illustrated by the example in Figure 6. For instance, if we choose $R(i-1, k, j) + R(i+1, k, j)$ as an auxiliary array in the first iteration of our algorithm, it cannot be compressed to a one-dimensional array because its reuses are along the outer loop dimension j and k . Furthermore, other solutions like $R(i-1, k-1, j) + R(i-1, k, j-1)$ are irregular and may result in discrete memory access, leading to poor locality. In contrast, if $R(i, k, j-1) + R(i, k, j+1)$ is chosen, it can be reused along i dimension.

IDF extraction is a greedy algorithm that prioritizes selecting expressions with zero delta in the innermost dimension, i.e., $e.exprDelta[i_n] = 0$. A zero delta in the innermost dimension indicates that the reuses are more likely to occur along the innermost i_n dimension, e.g., any binary subexpression of the `aa_1_0`'s $exprDelta[i] = 0$ and `aa_1_0` is reused along i loop in Figure 6. The algorithm runs in a try-until manner. First, it only generates the nodes with $exprDelta[i_n] = 0$ when constructing Pair Graph. This forms a subgraph of the original Pair Graph, containing significantly fewer nodes. If any solutions are found, the algorithm accepts them and proceeds to the next extraction iteration. If no solution is found, the algorithm will generate a new subgraph containing only nodes with $exprDelta[i_{n-1}] = 0$ and try to find the solution of this graph, repeating this process until it finds a solution at some dimension i .

Although IDF does not change the exponential worst-case time complexity $O(2^n)$, it speeds up solving by significantly reducing the number of nodes n in the Pair Graph. In the case of Figure 6, the original Pair Graph would contain more than one hundred nodes, making exhaustive search infeasible, whereas the IDF strategy reduces the search to a subgraph with only 18 nodes and solves it in negligible time. In addition, other heuristic or approximate methods for MIS could also be incorporated into the solver, but such extensions are beyond the scope of this paper.

8 Implementation and Limitations

We implemented RACE based on the Flang and Clang frontends in LLVM infrastructure. Our tool provides multiple options to control different levels of optimization aggressiveness. To invoke the transformation, developers insert compilation directives before and after the target loop nest, after which RACE performs a source-to-source transformation based on the abstract syntax tree (AST) produced by the frontend. RACE parses the annotated loop nests and extracts the candidate expression trees. Then it applies the proposed algorithm to detect redundant computations and rewrites the expression trees accordingly. After the detection finished, it compresses the auxiliary arrays and generates the transformed code based on the rewritten expression trees.

RACE currently targets a restricted but practically important class of loop nests. In this paper, we focus on perfectly nested loops without internal control flow, redundant computations over unmodified arrays, and relatively simple array references of the form $A[a_1 i_{s_1} + b_1] \cdots [a_n i_{s_n} + b_n]$. Some simple non-perfectly nested loops, such as those with initialization statements or with branches containing side-effect-free statements, may still be compatible with the RACE mechanism. However, extending the approach to general non-perfectly nested loops requires a complete legality analysis and formalization, which is left for future work.

RACE is an annotation-guided aggressive optimization. It uses developer annotations both to mark target loops and to assert semantic assumptions. For example, the method assumes that targeted function calls such as `sin` and `cos` are pure and deterministic, i.e., they have no side effects and always return the same floating-point value for the same input. Although compiler analyses may help check some of these assumptions, full automation remains difficult in the presence of issues such as pointer aliasing and unknown call targets. When reassociation is enabled, the optimization is provided on an "as-is" basis: arithmetic operations may be reordered, and numerical

stability is not guaranteed. It is therefore the developer’s responsibility to enable these aggressive modes only when such changes are acceptable.

Another limitation is the storage overhead of the auxiliary arrays introduced by the transformation. Although the contraction scheme in Section 6.2 can substantially reduce the footprint of temporary arrays, in extreme cases, the additional memory demand may exceed the available memory budget and lead to an out-of-memory failure. This extra storage may also increase memory traffic and cache pressure, so reducing arithmetic operations does not always lead to a speedup. Therefore, RACE is exposed as an opt-in optimization through annotations, so that programmers can decide whether the optimization is appropriate for a given loop.

9 Evaluation

9.1 Experiment Setup

We evaluated performance test in SPEC applications and extracted the kernels for more detailed performance analysis. The performance tests in SPEC applications were conducted on two different platforms: an Intel Xeon Gold 6467C 30-core CPU @ 3.40 GHz and an AMD EPYC 9654 96-core Processor @ 2.40 GHz. The detailed performance analysis of the extracted kernels was only conducted on the Intel platform due to the limited space in the paper and the similarity between the platforms. The SPEC applications were run using the standard configurations and input data provided by SPEC suite. The benchmarks were compiled by gfortran-9.4.0 or gcc-9.4.0 with compilation flags "-O3" to enable all standard optimizations.

9.2 Benchmark and Redundancy Analysis

Table 1. Benchmark optimization table. *Reduced Ops* is the fraction of the run-time reduced arithmetic operations(including sin/cos). *AA Num* is the number of auxiliary arrays found in total. *Alg Iter* is the iteration number of our algorithm to find all the auxiliary arrays. *Operations* are static operation analysis. RACE-NR refers to the use of our algorithm without reassociation.

App	Case	Reduced Ops	AA Num	Alg Iter	Operations(Base/RACE-NR/RACE)				
					add	sub	mul	div	sin/cos
POP	hdfft_gm	0.63	2	1	14/11/4	-	-	-	-
	calc_tpoints	0.55	9	3	9/9/6	-	11/5/5	-	16/4/4
	ocn_export	0.17	2	1	1/1/1	1/1/1	6/6/5	2/2/1	4/2/2
WRF	rhs_ph1	0.06	3	2	6/5/5	9/9/9	12/10/10	2/2/2	-
	rhs_ph2	0.16	3	2	6/5/5	9/9/9	12/10/10	2/2/2	-
	diffusion1	0.44	20	5	18/18/8	6/4/4	26/21/15	4/3/2	-
	diffusion2	0.60	19	5	18/16/8	6/4/4	26/20/14	4/3/2	-
	diffusion3	0.49	19	6	10/6/6	6/4/4	32/18/17	2/1/1	-
mgird	psinv	0.38	9	3	27/23/13	-	4/4/6	-	-
	resid	0.45	4	3	23/19/11	4/4/4	4/4/4	-	-
	rprj3	0.19	5	2	26/26/20	-	4/4/4	-	-
stencil	gaussian	0.43	13	4	24/24/16	-	25/6/11	1/1/1	-
	j3d27pt	0.35	20	3	26/26/18	-	27/15/15	1/1/1	-
	poisson	0.37	3	2	16/15/8	2/2/2	3/3/3	-	-
	derivative	0.71	86	11	99/54/45	96/24/16	297/101/76	-	-

We selected 11 cases in three different applications and 4 stencil kernels to evaluate the performance of the code generated by our tool. For each case above, RACE completes the source-to-source

transformation in less than 1.5 seconds, showing the efficiency of our algorithm. Table 1 shows the information of the selected cases. All of the loops in the cases are floating-point computations. The column label *Reduced Ops* represents reduction in run-time arithmetic operations. *Operations* column contains static analysis of operations, where the numbers are the counts of arithmetic operations in one iteration of the innermost loop.

POP is a high-performance ocean modeling software used for climate simulation and ocean circulation modeling. WRF is a numerical weather prediction model used for atmospheric research and operational forecasting. mgrid is a benchmark used to simulate multigrid algorithms for solving partial differential equations. The remaining cases are stencil kernels widely used in scientific applications. *Poisson*, *derivative* and *j3d27pt* are 3D stencil kernels whose grid size is set to 100^3 in our experiment, while *gaussian* are 2D stencil kernels with grid size set to 500^2 . The POP and WRF kernels were evaluated using their official implementations in the SPEC CPU 2017 benchmark suite, and the mgrid kernel was executed through the SPEC CPU 2000 suite.

Our algorithm effectively detects various redundant arithmetic operations. The sin/cos operations are reduced from 16 to only 4 in the case *calc_tpoints* of POP. A large number of auxiliary arrays are found in WRF's cases with few iterations, leading to nearly a half reduction in arithmetic operations. RACE can also handle large expression trees, such as the *derivative* case, where 86 auxiliary arrays are found and the multiplications are reduced from 297 to 76.

The detection algorithm on n-ary trees can more effectively identify redundancies. If only the algorithm on binary trees is used (which is denoted as RACE-NR in Table 1), the case *diffusion1* and *diffusion2* can only identify 8 and 9 auxiliary arrays, respectively, with a reduction of 8 and 11 redundant operations. In contrast, using the n-ary tree-based algorithm enables the identification of 20 and 19 auxiliary arrays, with a reduction of 25 and 26 redundant operations, respectively.

Cases in mgrid and stencil kernels involve many array references with the same rpi, leading to multiple solutions when solving the optimal reuse problem defined in Section 7.2. The inner-dimension-first method proposed in Section 7.3 identifies the auxiliary arrays with innermost loop reuse, which are more cache-friendly. In case *rprj3*, the loop variables inside the array subscripts are multiplied by a factor of 2. Although the names of the array references are the same, they have different rpi values. The redundancies are successfully detected, demonstrating the effectiveness of our two-level hash algorithm.

9.3 Speedup Results

We evaluated the single-core performance of the code generated by our tool. Results for SPEC benchmarks were measured within the applications using the standard SPEC run configuration. Since our algorithm targets loop-level optimization, we therefore report the loop-level speedup. Figure 7 and Figure 8 show the speedup compared to the baseline (original) code and ESR+ (ESR+reassociation) on the Intel and AMD CPUs, respectively.

Our results demonstrate that RACE achieves significant performance improvements across most benchmarks. RACE outperforms ESR+ in 14 out of 15 cases on both platforms, except for the *ocn_export* case where RACE generates the same code as ESR+. The RACE-NR method, despite not using reassociation, still provides notable improvements in cases like *calc_tpoints*, *derivative*. The ESR+ method shows several performance regressions. This is because the auxiliary variables extracted by ESR+ are scalars placed in the innermost loop and reused in a pipelined manner, which hinders the compiler's vectorization optimization. RACE can retain the auxiliary arrays as multi-dimensional arrays, which facilitates vectorization.

The overall performance gain is influenced by both the amount and types of detected redundancies, as detailed in Table 1. Fewer auxiliary arrays are identified in the *rhs_ph* case than in *diffusion*, explaining the lower speedup for *rhs_ph*. In contrast, the *derivative* case contains many redundant

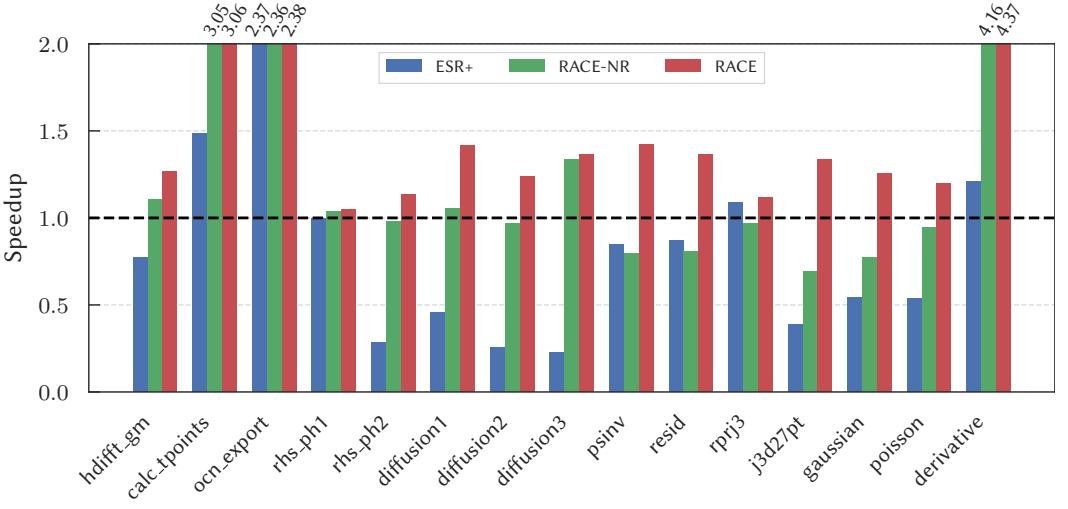


Fig. 7. The speedup of different optimization methods relative to the original code on the Intel CPU. Among the legend, RACE-NR refers to the use of our algorithm without reassociation; RACE uses the full abilities which involve reassociation, inner-dimension-first extraction, and array contraction. Note that the ESR+ represents ESR+reassociation.

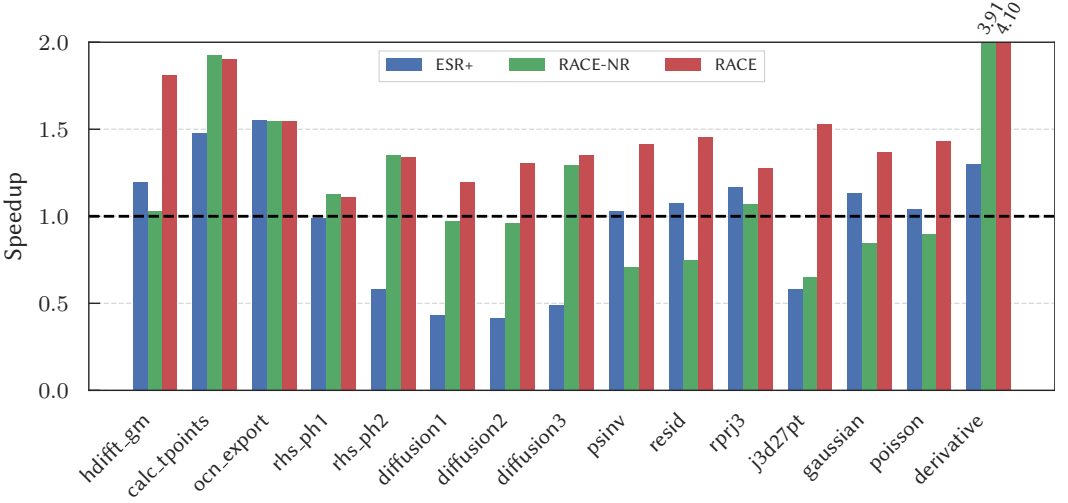


Fig. 8. The speedup of different optimization methods relative to the original code on the AMD CPU. The legend is the same as in Figure 7.

operations—up to 86 auxiliary arrays and 221 eliminated multiplications—leading to a significant speedup on both platforms. The intensity of redundant arithmetic operations largely determines the performance gain. On the Intel CPU, the speedup reaches $3.06\times$ in the *calc_tpoints* case of POP, where the sin/cos functions are computation-intensive. A similar pattern is observed in *ocn_export*, where the reduction in operations is small but the speedup remains notable.

However, for loops that are memory-access intensive, such as the *rprj3* case, the optimization is less effective. Moreover, reducing arithmetic intensity may increase memory pressure. While auxiliary arrays reduce arithmetic operations, they introduce additional memory allocations and increase the memory access footprint. The newly allocated arrays increase the cache working set, potentially leading to more cache misses. Compressing the dimensions of auxiliary arrays using the method described in Section 6.2 helps mitigate this issue.

9.4 Scaling Results

To gain deeper insights into the performance characteristics of the cases, we extracted the representative kernels from the SPEC applications and converted them into standalone benchmarks for detailed performance analysis. The performance analysis focuses on the runtime and memory data volume (the bytes transferred between the last-level cache and DRAM) for different problem sizes and the parallel scalability.

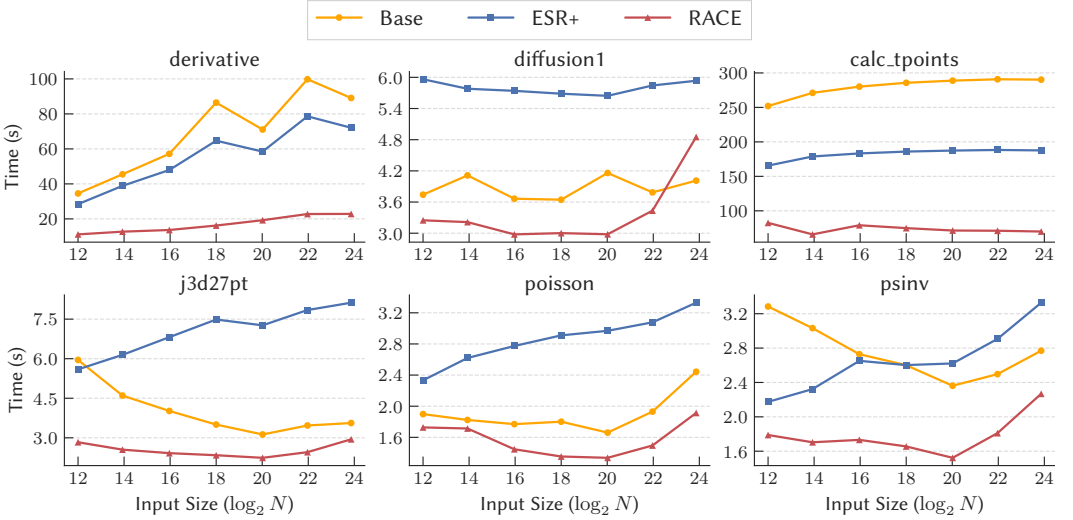


Fig. 9. Runtime vs. Input Size on Intel CPU. The total computation amounts ($N \times T = 2^{31}$) are the same for different kernels and input sizes.

9.4.1 Runtime Comparison. Figure 9 presents the runtime of six representative kernels on the Intel CPU for varying input sizes ($\log_2 N$). For all experiments, the product of input size and time step ($N \times T = 2^{31}$) is fixed to ensure a fair comparison across different kernels and input sizes.

Overall, RACE achieves the lowest runtime, except for *diffusion1* at size $N = 2^{24}$, consistent with the previous results. The runtime trends vary across different kernels. For compute-intensive kernels such as *derivative* and *calc_tpoints*, the runtime of RACE increases more slowly as problem size increases. In contrast, for less compute-intensive kernels such as *diffusion1* and *psinv*, the runtime grows more steeply at larger input sizes, suggesting that the memory access cost becomes dominant, which conceals part of the performance gain from reducing arithmetic operations.

Although the total computation amounts are kept the same, the runtime of certain kernels can be shorter at medium input sizes than at small input sizes, due to the overhead of handling loop remainder from vectorization, which is more pronounced when the input grid is small.

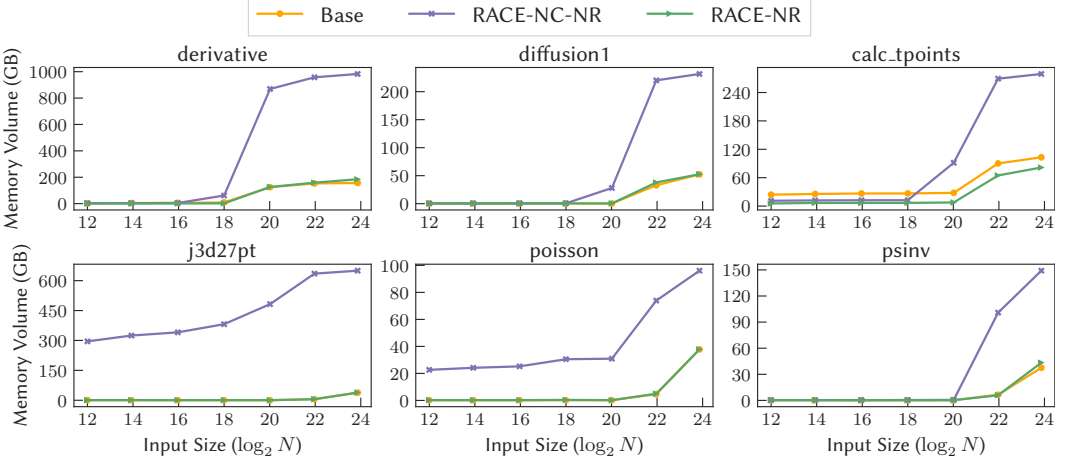


Fig. 10. Memory Volume vs. Input Size on Intel CPU. RACE-NC-NR denotes the RACE method without array contraction and reassociation. RACE-NR uses array contraction while remaining reassociation disabled.

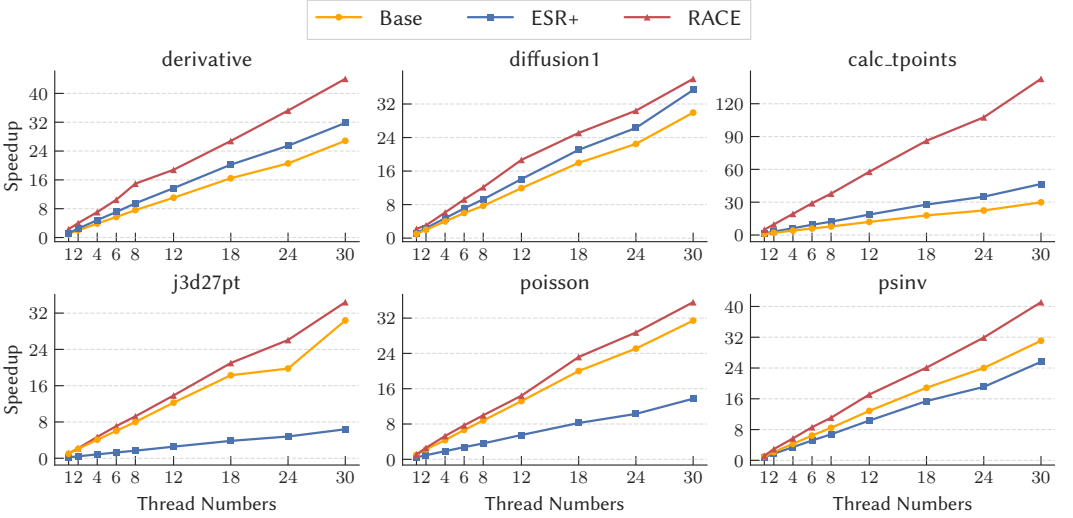


Fig. 11. Multithreaded Scalability on Intel CPU. The speedup is computed as $t_{1, \text{Base}}/t_{n, \text{opt}}$, where $t_{n, \text{opt}}$ denotes the runtime of opt-optimized code using n threads. $t_{1, \text{Base}}$ is the runtime of the original code using one thread. All kernel and thread number configurations were run with $N = 180^3$, $T = 500$.

9.4.2 Memory Volume Saved by Array Contraction. The run-time memory volume is measured to demonstrate the effectiveness of array contraction proposed in Section 6.2. Figure 10 shows the memory volume transferred between the last-level cache and DRAM for different input sizes on the Intel CPU. RACE-NC-NR denotes the RACE method without array contraction and reassociation. The only difference between RACE-NC-NR and RACE-NR is that the former disables array contraction while the latter enables it. This allows us to isolate the impact of array contraction from other factors affecting memory volume.

The results show that the memory volume of RACE-NC-NR is significantly higher than that of RACE-NR, while RACE-NR is close to the baseline code. This indicates that array contraction effectively alleviates the memory pressure introduced by auxiliary arrays, especially at larger input sizes, leading to substantial performance improvements. The memory volume is much higher at large input sizes because the input grids exceed the cache capacity.

9.4.3 Multi-threaded Scalability. We also evaluate the OpenMP multithreaded scalability of our approach, which is shown in Figure 11. The speedup is compared against the baseline code using a single thread. The results indicate that RACE achieves nice scalability and outperforms the other methods consistently across different thread counts. The introduced auxiliary arrays do not hinder multi-threaded performance.

10 Conclusion

This paper proposes RACE, a two-level hashing framework for detecting redundant array computations in nested loops. The first-level hash uses rpi to determine whether two array references share the same reference pattern. The second-level hash computes eri for each binary expression as an attribute of the expression, encoding expression-level reuse without explicitly comparing different expressions. Using eri, we give a linear-time algorithm on binary expression trees and discuss code generation and optimization for the resulting auxiliary arrays. Furthermore, tree flattening and the Pair Graph are introduced to identify redundancies in n-ary trees. The problem is formulated and reduced to a maximum independent set problem. To efficiently find a solution, we propose a memory-friendly inner-dimension-first heuristic that prioritizes selecting auxiliary arrays with inner-loop reuse. Experiments show that RACE achieves significant speedups across various benchmarks, demonstrating the effectiveness of the algorithm.

Acknowledgments

This work is supported by National Key R&D Program of China under Grant No. 2025YFB3003604, 2023YFB3001700, National Natural Science Foundation of China under Grant No. 62372432.

Data-Availability Statement

The artifact for the experiments is available on Zenodo [29]. It is provided as a Docker image archive that includes all programs and scripts required to reproduce the experimental results.

References

- [1] Abu-Sufah, Kuck, and Lawrie. 1981. On the Performance Enhancement of Paging Systems Through Program Analysis and Transformations. *IEEE Trans. Comput.* C-30, 5 (1981), 341–356. doi:10.1109/TC.1981.1675792
- [2] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. 2006. *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Addison-Wesley Longman Publishing Co., Inc., USA.
- [3] J. R. Allen, Ken Kennedy, Carrie Porterfield, and Joe Warren. 1983. Conversion of control dependence to data dependence. In *Proceedings of the 10th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (Austin, Texas) (POPL '83). Association for Computing Machinery, New York, NY, USA, 177–189. doi:10.1145/567067.567085
- [4] Utpal Banerjee. 1979. *Speedup of ordinary programs*. Ph.D. Dissertation. USA. AAI8008967.
- [5] Protonu Basu, Mary Hall, Samuel Williams, Brian Van Straalen, Leonid Oliker, and Phillip Colella. 2015. Compiler-Directed Transformation for Higher-Order Stencils. In *2015 IEEE International Parallel and Distributed Processing Symposium*. IEEE, Hyderabad, 313–323. doi:10.1109/IPDPS.2015.103
- [6] David Callahan, Steve Carr, and Ken Kennedy. 1990. Improving register allocation for subscripted variables. In *Proceedings of the ACM SIGPLAN 1990 conference on Programming language design and implementation - PLDI '90*. ACM Press, White Plains, New York, United States, 53–65. doi:10.1145/93542.93553
- [7] Thomas J. Watson IBM Research Center, F.E. Allen, and J. Cocke. 1971. *A Catalogue of Optimizing Transformations*. IBM Thomas J. Watson Research Center.

- [8] Yuze Chi and Jason Cong. 2020. Exploiting Computation Reuse for Stencil Accelerators. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, San Francisco, CA, USA, 1–6. doi:10.1109/DAC18072.2020.9218680
- [9] P. Clauss and V. Loechner. 1996. Parametric analysis of polyhedral iteration spaces. In *Proceedings of International Conference on Application Specific Systems, Architectures and Processors: ASAP '96*. 415–424. doi:10.1109/ASAP.1996.542833
- [10] John Cocke. 1969. *Programming languages and their compilers: Preliminary notes*. New York University, USA.
- [11] John Cocke. 1970. Global common subexpression elimination. *SIGPLAN Not.* 5, 7 (July 1970), 20–24. doi:10.1145/390013.808480
- [12] Keith Cooper, Jason Eckhardt, and Ken Kennedy. 2008. Redundancy Elimination Revisited. In *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques*. ACM, Toronto Ontario Canada, 12–21. doi:10.1145/1454115.1454120
- [13] K. D. Cooper and K. Kennedy. 1989. Fast interprocedural alias analysis. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (Austin, Texas, USA) (POPL '89)*. Association for Computing Machinery, New York, NY, USA, 49–59. doi:10.1145/75277.75282
- [14] Steven J. Deitz, Bradford L. Chamberlain, and Lawrence Snyder. 2001. Eliminating Redundancies in Sum-of-Product Array Computations. In *Proceedings of the 15th International Conference on Supercomputing*. ACM, Sorrento Italy, 65–77. doi:10.1145/377792.377807
- [15] Yufei Ding and Xipeng Shen. 2017. GLORE: Generalized Loop Redundancy Elimination upon LER-notation. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (Oct. 2017), 1–28. doi:10.1145/3133898
- [16] Amer Diwan, Kathryn S. McKinley, and J. Eliot B. Moss. 1998. Type-based alias analysis. In *Proceedings of the ACM SIGPLAN 1998 Conference on Programming Language Design and Implementation (Montreal, Quebec, Canada) (PLDI '98)*. Association for Computing Machinery, New York, NY, USA, 106–117. doi:10.1145/277650.277670
- [17] Ken Kennedy and John R. Allen. 2001. *Optimizing compilers for modern architectures: a dependence-based approach*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.
- [18] Jens Knoop, Oliver Rüthing, and Bernhard Steffen. 1994. Partial dead code elimination. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation (Orlando, Florida, USA) (PLDI '94)*. Association for Computing Machinery, New York, NY, USA, 147–158. doi:10.1145/178243.178256
- [19] Jens Knoop, Oliver Rüthing, and Bernhard Steffen. 2004. Lazy code motion. *SIGPLAN Not.* 39, 4 (April 2004), 460–472. doi:10.1145/989393.989439
- [20] Stefan Kronawitter, Sebastian Kuckuk, and Christian Lengauer. 2016. Redundancy Elimination in the ExaStencils Code Generator. In *Algorithms and Architectures for Parallel Processing*, Jesus Carretero, Javier Garcia-Blas, Victor Gergel, Vladimir Voevodin, Iosif Meyerov, Juan A. Rico-Gallego, Juan C. Díaz-Martín, Pedro Alonso, Juan Durillo, José Daniel García Sánchez, Alexey L. Lastovetsky, Fabrizio Marozzo, Qin Liu, Zakirul Alam Bhuiyan, Karl Furlinger, Josef Weidendorfer, and José Gracia (Eds.). Vol. 10049. Springer International Publishing, Cham, 159–173. doi:10.1007/978-3-319-49956-7_13
- [21] Leslie Lamport. 1974. The parallel execution of DO loops. *Commun. ACM* 17, 2 (Feb. 1974), 83–93. doi:10.1145/360827.360844
- [22] Fabio Luporini, David A. Ham, and Paul H. J. Kelly. 2018. An Algorithm for the Optimization of Finite Element Integration Loops. *ACM Trans. Math. Software* 44, 1 (March 2018), 1–26. doi:10.1145/3054944
- [23] Fabio Luporini, Mathias Louboutin, Michael Lange, Navjot Kukreja, Philipp Witte, Jan Hückelheim, Charles Yount, Paul H. J. Kelly, Felix J. Herrmann, and Gerard J. Gorman. 2020. Architecture and Performance of Devito, a System for Automated Stencil Computation. *ACM Trans. Math. Software* 46, 1 (March 2020), 1–28. doi:10.1145/3374916
- [24] Dror E. Maydan, John L. Hennessy, and Monica S. Lam. 1991. Efficient and exact data dependence analysis. In *Proceedings of the ACM SIGPLAN 1991 Conference on Programming Language Design and Implementation (Toronto, Ontario, Canada) (PLDI '91)*. Association for Computing Machinery, New York, NY, USA, 1–14. doi:10.1145/113445.113447
- [25] E. Morel and C. Rennoise. 1979. Global optimization by suppression of partial redundancies. *Commun. ACM* 22, 2 (Feb. 1979), 96–103. doi:10.1145/359060.359069
- [26] Steven S. Muchnick. 1998. *Advanced compiler design and implementation*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.
- [27] M. Potkonjak, M.B. Srivastava, and A.P. Chandrakasan. 1996. Multiple constant multiplications: efficient and versatile framework and algorithms for exploring common subexpression elimination. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 15, 2 (Feb. 1996), 151–165. doi:10.1109/43.486662
- [28] Pengfei Su, Shasha Wen, Hailong Yang, Milind Chabbi, and Xu Liu. 2019. Redundant Loads: A Software Inefficiency Indicator. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, Montreal, QC, Canada, 982–993. doi:10.1109/ICSE.2019.00103
- [29] Zixuan Wang, Liang Yuan, Xianmeng Jiang, Kun Li, Junmin Xiao, and Yunquan Zhang. 2026. Artifact Evaluation for "Redundant Array Computation Elimination". Zenodo. doi:10.5281/zenodo.19563382

- [30] Shasha Wen, Xu Liu, and Milind Chabbi. 2015. Runtime Value Numbering: A Profiling Technique to Pinpoint Redundant Computations. In *2015 International Conference on Parallel Architecture and Compilation (PACT)*. 254–265. doi:10.1109/PACT.2015.29
- [31] Michael Joseph Wolfe. 1978. *Techniques for improving the inherent parallelism in programs*. Number 929. Department of Computer Science, University of Illinois at Urbana-Champaign.
- [32] Hongtao Yu and Zhiyuan Li. 2012. Fast loop-level data dependence profiling. In *Proceedings of the 26th ACM International Conference on Supercomputing* (San Servolo Island, Venice, Italy) (*ICS '12*). Association for Computing Machinery, New York, NY, USA, 37–46. doi:10.1145/2304576.2304584
- [33] Lufeng Yuan, Junhong Liu, Yulong Luo, and Guangming Tan. 2016. Locality of Computation for Stencil Optimization. In *Algorithms and Architectures for Parallel Processing*, Jesus Carretero, Javier Garcia-Blas, Ryan K.L. Ko, Peter Mueller, and Koji Nakano (Eds.). Vol. 10048. Springer International Publishing, Cham, 449–456. doi:10.1007/978-3-319-49583-5_34

A Proof of Theorem 7.1

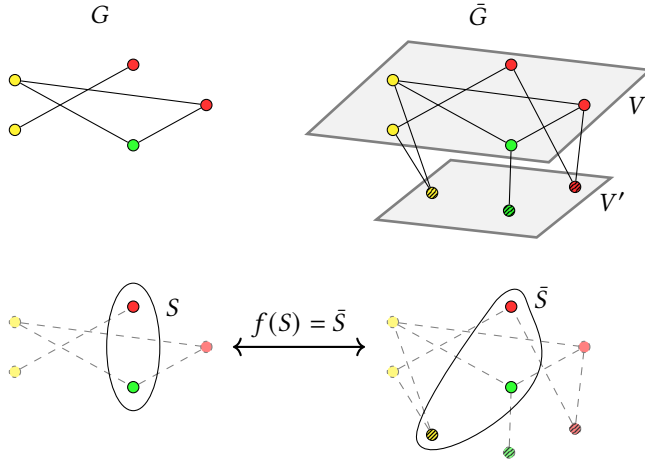


Fig. 12. The above two graphs illustrate an example of the graph G and the corresponding auxiliary graph $\bar{G}$. The bottom two graphs highlight an independent set S of G and its corresponding independent set $\bar{S} = f(S)$ of $\bar{G}$.

The objective function defined in Equation 1 is

$$\operatorname{argmax}_{S \in I_G} |S| - |\operatorname{eri}(S)|,$$

where I_G denotes the set of independent sets of graph G .

To reduce this problem to a Maximum Independent Set (MIS) problem, an auxiliary graph $\bar{G} = (\bar{V}, \bar{E})$ is constructed. For each eri value k , $\bar{G}$ adds an auxiliary node $\bar{v}_k$ connected to all nodes with eri value k in G , i.e., $\bar{V} = V \cup V'$ where $V' = \{\bar{v}_k \mid k \in \operatorname{eri}(V)\}$, $\operatorname{eri}(\bar{v}_k) = k$, and $\bar{E} = E \cup \{(\bar{v}_k, v) \mid \bar{v}_k \in V', v \in V, \operatorname{eri}(v) = k\}$. An example of graph G and the corresponding auxiliary graph $\bar{G}$ are shown in Figure 12. The formal reduction description is in Theorem 7.1. To prove the theorem, we first introduce the following function and lemmas.

Define transformation function $f : I_G \rightarrow f(I_G)$ as

$$f(S) = S \cup \{\bar{v}_k \in V' \mid k \notin \operatorname{eri}(S)\}.$$

We have lemma A.1.

LEMMA A.1. f is a bijective function.

PROOF. It can be seen that f adds auxiliary nodes of $\bar{G}$ with disappeared eri values into S . We have $\forall S_0, S_1 \in I_G, S_0 \neq S_1 \implies f(S_0) \neq f(S_1)$. So f is injective. The codomain of f is its range $f(I_G)$, so it is surjective. Therefore, f is bijective. $\square$

Figure 12 shows how f maps an independent set S of G to an independent set $\bar{S} = f(S)$ of $\bar{G}$ by adding auxiliary nodes into S . Lemma A.2 is stated as follows, which allows f to connect the solutions of the original problem on G with the solutions of the MIS problem on $\bar{G}$.

LEMMA A.2. *Any MIS of $\bar{G}$ is in the range of function f .*

PROOF. According to the construction of $\bar{G}$, $\forall \bar{S} \in f(I_G)$, $\bar{S} \in I_{\bar{G}}$ (i.e., $\bar{S}$ is an independent set of $\bar{G}$), so we have $f(I_G) \subseteq I_{\bar{G}}$. Actually, we can prove that $f(I_G)$ is the set of all the independent sets $\bar{S}$ of $\bar{G}$ such that $\text{eri}(\bar{S}) = \text{eri}(V)$, i.e., $f(I_G) = \{\bar{S} \in I_{\bar{G}} \mid \text{eri}(\bar{S}) = \text{eri}(V)\}$.

$\subseteq$: given $\bar{S} \in f(I_G)$, there exists $S \in I_G$ such that $f(S) = \bar{S}$. The function f augments the independent set S with $\bar{v}_k$ for all k values that are not in $\text{eri}(S)$, thus we have $\text{eri}(f(S)) = \text{eri}(\bar{S}) = \text{eri}(V)$.

$\supseteq$: given an independent set $\bar{S} \in I_{\bar{G}}$ such that $\text{eri}(\bar{S}) = \text{eri}(V)$, we have $S = \bar{S} \setminus V'$ is an independent set of G and $\bar{S} = f(S) \in f(I_G)$.

It's easy to see that a MIS $\bar{S}$ of $\bar{G}$ satisfying $\text{eri}(\bar{S}) = \text{eri}(V)$ (otherwise some $\bar{v}_k$ could be added without breaking independence, contradicting maximality). Thus, $f(I_G)$ contains all MIS of $\bar{G}$. $\square$

PROOF OF THEOREM 7.1. Define objective functions

$$F(S) = |S| + |\text{eri}(V)| - |\text{eri}(S)|, \quad \text{and} \quad \bar{F}(S) = |S|.$$

Since $|\text{eri}(V)|$ is a constant given G , optimizing $F(S)$ is equivalent to optimizing $F(S) - |\text{eri}(V)|$ which is the objective function in Equation 1, as additive constants do not affect the optimal solution. We now prove that solving the original problem defined by Equation 1 ($\arg\max_{S \in I_G} F(S)$) on graph G is equivalent to solving the MIS problem ($\arg\max_{\bar{S} \in I_{\bar{G}}} \bar{F}(\bar{S})$) on the auxiliary graph $\bar{G}$:

$f(S)$ adds auxiliary nodes with disappeared eri values into S . So we have

$$\begin{aligned} \forall S \in I_G, \bar{F}(f(S)) &= |f(S)| \\ &= |S| + |\text{eri}(V)| - |\text{eri}(S)| \\ &= F(S). \end{aligned} \tag{2}$$

As proved in Lemma A.1, f is bijective, so we can rewrite Equation 2 as

$$\forall \bar{S} \in f(I_G), \bar{F}(\bar{S}) = F(f^{-1}(\bar{S})). \tag{3}$$

Since Lemma A.2 shows that any MIS of $\bar{G}$ belongs to $f(I_G)$, the objective function for the MIS problem on $\bar{G}$ is equivalent to $\arg\max_{\bar{S} \in f(I_G)} \bar{F}(\bar{S})$. Let $\bar{S}^*$ denotes one of the solutions for the MIS problem on $\bar{G}$ and $S^* = f^{-1}(\bar{S}^*)$. Then we have

$$\begin{aligned} F(S^*) &= F(f^{-1}(\bar{S}^*)) \\ &= \bar{F}(\bar{S}^*) \quad (\text{by Equation 3}) \\ &= \max_{\bar{S} \in f(I_G)} \bar{F}(\bar{S}) \quad (\text{because } \bar{S}^* \text{ is an MIS on } \bar{G}) \\ &= \max_{S \in I_G} \bar{F}(f(S)) \quad (\text{replace variable } \bar{S} \text{ with } f(S)) \\ &= \max_{S \in I_G} F(S) \quad (\text{by Equation 2}). \end{aligned}$$

So $S^* = \arg\max_{S \in I_G} F(S)$. Thus, any solution $\bar{S}^*$ for the MIS problem on $\bar{G}$ yields a solution $S^* = f^{-1}(\bar{S}^*)$ for the optimization problem on G defined by Equation 1. $\square$